ROZDZIAŁ 5.

Maszyny wektorów nośnych

Maszyna wektorów nośnych (ang. *support vector machine* — SVM) stanowi potężny i wszechstronny model uczenia maszynowego, zdolny do przeprowadzania klasyfikacji liniowej, nieliniowej, regresji, a nawet do wykrywania elementów odstających. Jest to jedno z najpopularniejszych rozwiązań w świecie uczenia maszynowego i każdy analityk danych powinien potrafić się nim posługiwać. Maszyny SVM przydają się zwłaszcza do klasyfikowania złożonych, małych lub średnich zbiorów danych.

W tym rozdziale poznasz podstawowe koncepcje związane z maszynami SVM, ich mechanizm działania oraz metody korzystania z nich.

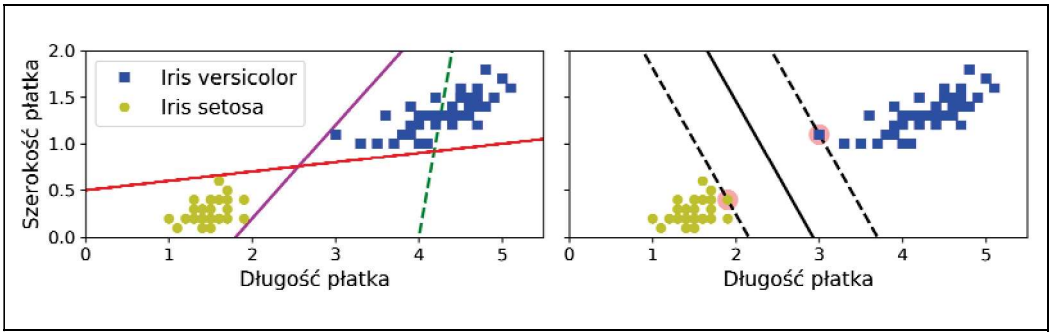
Liniowa klasyfikacja SVM

Podstawy działania maszyn SVM najlepiej wyjaśnić wizualnie. Rysunek 5.1 prezentuje część zbioru danych Iris, którym zajmowaliśmy się pod koniec rozdziału 4. Możemy wyraźnie rozdzielić dwie klasy za pomocą prostej (są one **liniowo rozdzielne/separowalne**). Na lewym wykresie widzimy granice decyzyjne trzech możliwych klasyfikatorów liniowych. Model, którego granica decyzyjna jest symbolizowana linią przerywaną, jest tak zły, że nawet nie rozdziela prawidłowo klas. Pozostałe dwa modele są znakomicie dopasowane do tego zbioru uczącego, ale ich granice decyzyjne znajdują się tak blisko przykładów, że prawdopodobnie nie będą spisywały się dobrze wobec nowych danych. Z kolei na prawym wykresie widzimy granicę decyzyjną klasyfikatora SVM; widoczna linia nie tylko rozdziela obydwie klasy, ale także utrzymuje możliwie duży dystans od najbliższej próbki. Możemy traktować maszynę SVM jako wyznaczanie najszerszej możliwej „ulicy" (symbolizowanej liniami przerywanymi) pomiędzy klasami. Proces ten nazywamy **klasyfikowaniem maksymalnego marginesu** (ang. *large margin classification*).

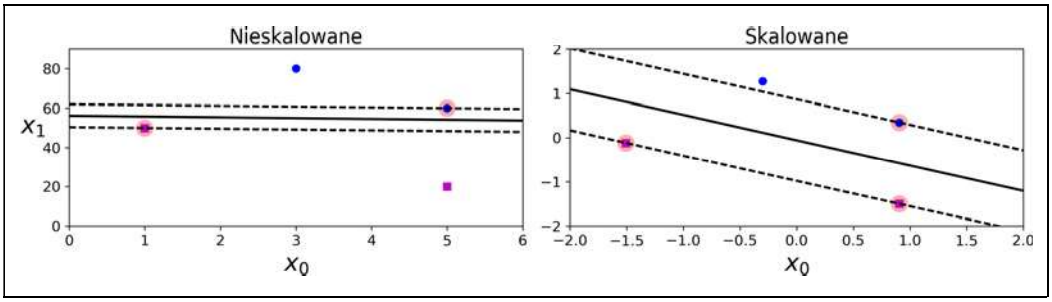
Zwróć uwagę, że dodawanie kolejnych przykładów znajdujących się na zewnątrz marginesu w żaden sposób nie wpłynie na granicę decyzyjną: jest całkowicie określona („noszona") przez próbki znajdujące się na krańcach „ulicy". Przykłady te są nazywane **wektorami nośnymi** (ang. *support vectors*; są one ujęte w kółka na rysunku 5.1).



Maszyny SVM są czułe na skale cech, o czym możemy się przekonać, patrząc na rysunek 5.2: na lewym wykresie skala w osi pionowej jest znacznie większa od skali w osi poziomej, dlatego najszersza możliwa „ulica" jest ułożona niemal horyzontalnie. Po przeskalowaniu cech (np. za pomocą klasy `StandardScaler`) granica decyzyjna na prawym wykresie prezentuje się znacznie lepiej.



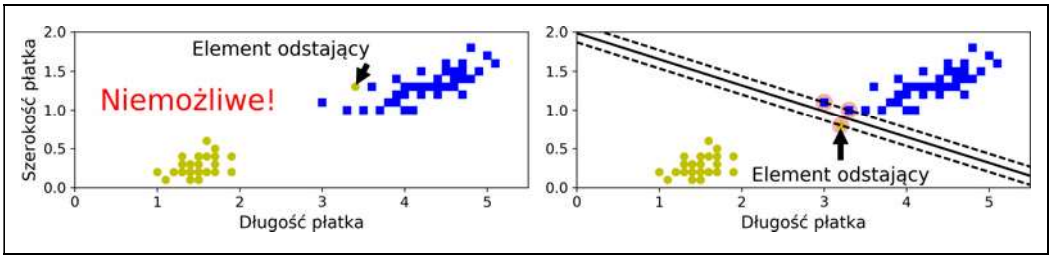
Rysunek 5.1. Klasyfikowanie maksymalnego marginesu



Rysunek 5.2. Czułość na skalę cech

Klasyfikacja miękkiego marginesu

Jeśli rygorystycznie wymusimy, aby wszystkie znajdowały się poza „ulicą" i po prawej stronie wykresu, to będziemy mieli do czynienia z **klasyfikacją twardego marginesu** (ang. *hard margin classification*). Rozwiązanie to cechują dwa problemy. Po pierwsze, działa ono jedynie wtedy, gdy dane są liniowo rozdzielne. Po drugie, jest ono wrażliwe na elementy odstające. Rysunek 5.3 przedstawia zestaw danych *Iris* z umieszczonym tylko jednym elementem odstającym; z tego powodu na lewym wykresie znalezienie twardego marginesu okazuje się niemożliwe, prawy wykres ukazuje zupełnie odmienną granicę decyzyjną od widocznej na rysunku 5.1 — oznacza to, że model prawdopodobnie nie będzie zbyt dobrze generalizował wyników.

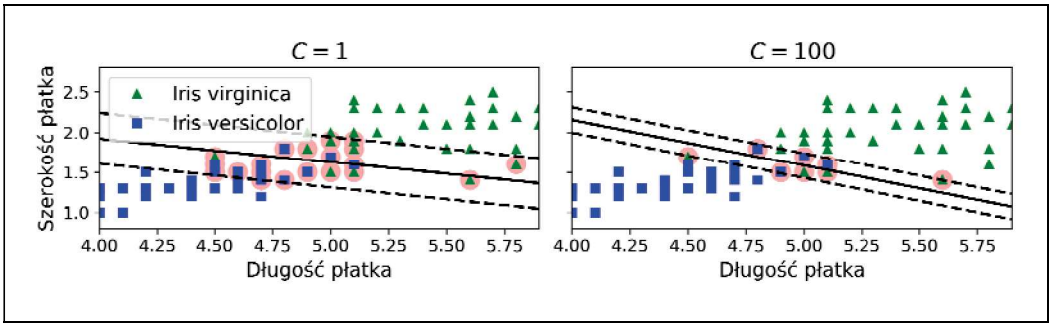


Rysunek 5.3. Wrażliwość klasyfikacji twardego marginesu na elementy odstające

Aby uniknąć tych problemów, skorzystaj z elastyczniejszego modelu. Naszym celem jest znalezienie dobrego kompromisu pomiędzy jak największą szerokością „ulicy" a ograniczeniem **naruszania margine-**

su (ang. *margin violation*; np. próbki przekraczające granice marginesu lub nawet trafiające na jego niewłaściwą stronę). Jest to tzw. **klasyfikacja miękkiego marginesu** (ang. *soft margin classification*).

Podczas tworzenia modelu SVM za pomocą modułu Scikit-Learn możemy wyznaczyć liczbę hiperparametrów. Jednym z nich jest hiperparametr C. Jeżeli wprowadzimy jego małą wartość, to uzyskamy model widoczny po lewej stronie na rysunku 5.4. W przypadku dużej wartości hiperparametru C otrzymamy model pokazany po prawej stronie. Przekroczenia marginesów są złe. Zazwyczaj staramy się, żeby było ich jak najmniej. W tym przypadku jednak model po lewej stronie ma wiele naruszeń marginesów, ale prawdopodobnie będzie cechował się lepszym uogólnianiem.



Rysunek 5.4. *Mniejsza liczba naruszeń marginesu (po prawej) a większa szerokość marginesu (po lewej)*



Jeżeli Twój model ulega przetrenowaniu, możesz go regularyzować za pomocą hiperparametru C.

Poniższy fragment kodu wczytuje zbiór danych Iris, skaluje cechy, a następnie trenuje liniowy model SVM (korzystamy z klasy `LinearSVC`, hiperparametru `C=1` i **zawiasowej funkcji straty** — ang. *hinge loss* — do której niebawem powrócimy) do rozpoznawania kwiatów odmiany *Iris virginica*:

```
import numpy as np
from sklearn import datasets
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

iris = datasets.load_iris()
X = iris["data"][:, (2, 3)] # Długość płatka, szerokość płatka
y = (iris["target"] == 2).astype(np.float64) # Iris virginica

svm_clf = Pipeline([
        ("scaler", StandardScaler()),
        ("linear_svc", LinearSVC(C=1, loss="hinge")),
    ])

svm_clf.fit(X, y)
```

Otrzymany model jest widoczny po prawej stronie na rysunku 5.4.

Jak zwykle możemy użyć wyuczonego modelu do wyliczenia prognoz:

```
>>> svm_clf.predict([[5.5, 1.7]])
array([1.])
```



Klasyfikatory SVM, w przeciwieństwie do klasyfikatorów regresyjnych, nie generują prawdopodobieństw przynależności próbki do poszczególnych klas.

Zamiast korzystać z klasy `LinearSVC` możemy użyć klasy `SVC` z wyznaczonym jądrem liniowym. Podczas tworzenia modelu SVC możemy napisać `SVC(kernel="linear", C=1)`. Ewentualnie jesteśmy w stanie zaimplementować klasę `SGDClassifier` — `SGDClassifier(loss="hinge", alpha=1/(m*C))`. Wykorzystujemy w tym przypadku algorytm stochastycznego spadku wzdłuż gradientu (zob, rozdział 4.) do wytrenowania liniowego klasyfikatora SVM. Metoda ta osiąga zbieżność wolniej niż klasa `LinearSVC`, ale przydaje się do przetwarzania zadań klasyfikacji przyrostowej lub olbrzymich zbiorów danych, które nie mieszczą się w pamięci (uczenie pozakorowe).



Klasa `LinearSVC` regularyzuje punkt obciążenia, dlatego należy najpierw wyśrodkować zbiór uczący, odejmując od niego wartość średnią. Jeżeli skalujemy dane za pomocą klasy `StandardScaler`, proces ten jest przeprowadzany automatycznie. Wyznacza również wartość `hinge` w hiperparametrze `loss`, gdyż nie jest ona domyślna. Na koniec, uzyskasz lepszą wydajność, jeśli wstawisz wartość `False` do hiperparametru `dual`, chyba że w zestawie uczącym znajduje się więcej cech niż przykładów (w dalszej części rozdziału przyjrzymy się uważniej dualności).

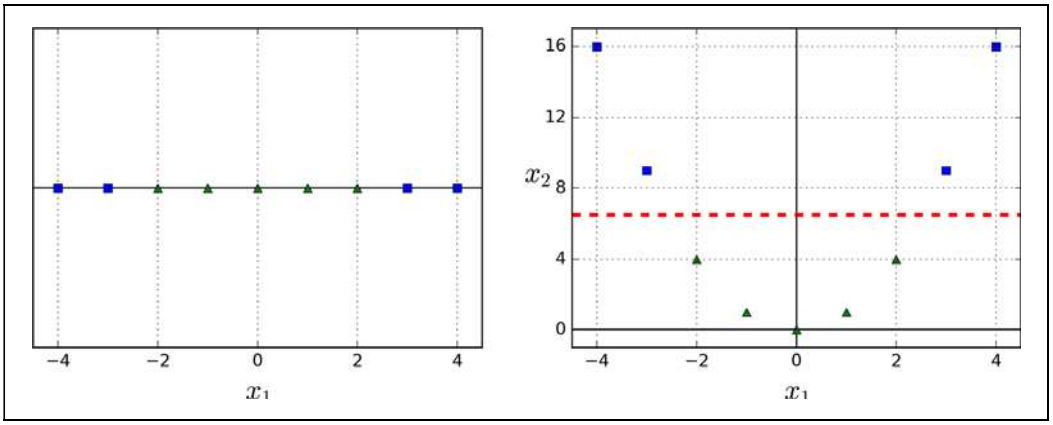
Nieliniowa klasyfikacja SVM

Liniowe klasyfikatory SVM są skuteczne i w wielu przypadkach sprawdzają się zdumiewająco dobrze, jednak wiele zbiorów danych nawet w części nie jest liniowo rozdzielnych. Jednym ze sposobów na radzenie sobie z takimi nieliniowymi zbiorami danych jest dodawanie kolejnych (np. wielomianowych; zob. rozdział 4.); w niektórych przypadkach możemy uzyskać w ten sposób zestaw liniowo rozdzielnych danych. Spójrz na lewy wykres na rysunku 5.5: widzimy na nim prosty zbiór danych zawierający tylko jedną cechę x_1. Łatwo zauważyć, że nie jest on liniowo rozdzielny. Jeśli jednak dodamy drugą cechę, $x_2 = (x_1)^2$, wynikowy dwuwymiarowy wykres stanie się idealnie separowalny liniowo.

Aby zaimplementować tę koncepcję w module Scikit-Learn, stwórz potok (`Pipeline`) zawierający transformator `PolynomialFeatures` (omówiony w rozdziale 4., w podrozdziale „Regresja wielomianowa”) oraz kolejno klasy `StandardScaler` i `LinearSVC`. Przetestujmy to rozwiązanie na zestawie danych sierpowatych (`moons`): jest to testowy zestaw danych wykorzystywany w klasyfikacji binarnej, w którym punkty danych tworzą kształt dwóch nachodzących na siebie półksiężyców (rysunek 5.6). Tworzymy ten zestaw danych za pomocą funkcji make_moons():

```
from sklearn.datasets import make_moons
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures

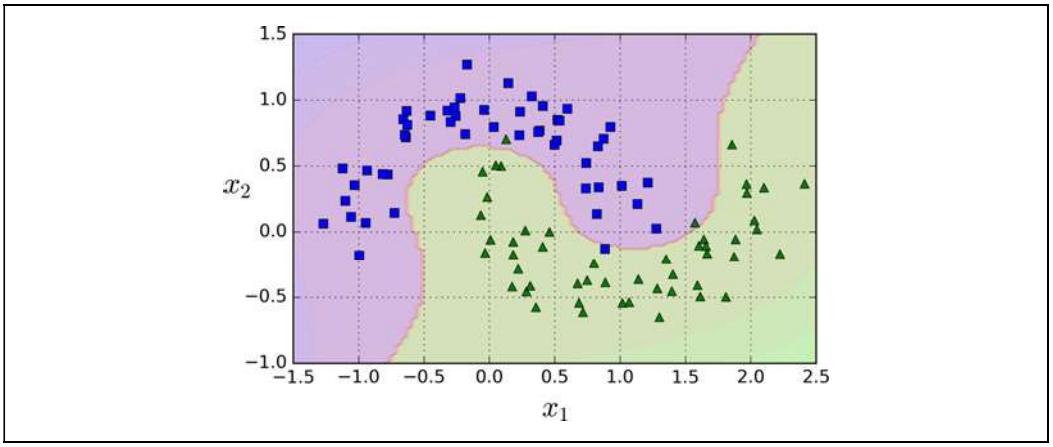
X, y = make_moons(n_samples=100, noise=0.15)
polynomial_svm_clf = Pipeline([
        ("poly_features", PolynomialFeatures(degree=3)),
        ("scaler", StandardScaler()),
```



Rysunek 5.5. Dodawanie cech w celu uzyskania zbioru rozdzielnych liniowo danych

```
        ("svm_clf", LinearSVC(C=10, loss="hinge"))
    ])

polynomial_svm_clf.fit(X, y)
```



Rysunek 5.6. Liniowy klasyfikator SVM wykorzystujący cechy wielomianowe

Jądro wielomianowe

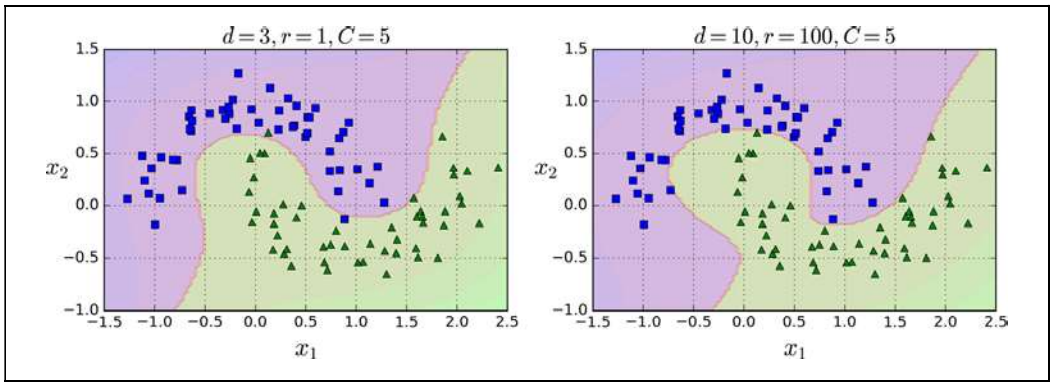
Dodawanie wielomianowych cech jest rozwiązaniem bardzo prostym do zaimplementowania i świetnie sprawującym się z różnego rodzaju algorytmami uczenia maszynowego (nie tylko z maszynami SVM). Pomimo tego metoda ta przy wielomianach niższego stopnia nie radzi sobie zbyt dobrze ze złożonymi zbiorami danych, natomiast w przypadku wielomianów dużego stopnia generuje olbrzymią liczbę cech, co znacznie spowalnia działanie modelu.

Na szczęście podczas stosowania maszyn SVM możemy wprowadzić niemal cudowną technikę matematyczną, zwaną **sztuczką z funkcją jądra** (ang. *kernel trick*; niebawem zostanie ona dokładniej wyjaśniona). Dzięki niej jesteśmy w stanie uzyskiwać takie same wyniki, jak po dodaniu wielu cech wielomianowych, nawet jeśli są nimi wielomiany dużego stopnia, bez potrzeby ich generowania.

Zatem nie musimy obawiać się gwałtownego wzrostu liczby cech, gdyż w rzeczywistości wcale ich nie dodajemy. Sztuczka ta jest zaimplementowana w klasie `SVC`. Sprawdźmy jej działanie na znanym już nam zbiorze danych sierpowatych:

```
from sklearn.svm import SVC
poly_kernel_svm_clf = Pipeline([
        ("scaler", StandardScaler()),
        ("svm_clf", SVC(kernel="poly", degree=3, coef0=1, C=5))
    ])
poly_kernel_svm_clf.fit(X, y)
```

Uczymy tu klasyfikator SVM za pomocą jądra wielomianowego trzeciego stopnia. Odzwierciedla to lewy wykres na rysunku 5.7. Na prawym wykresie widzimy z kolei inny klasyfikator SVM wykorzystujący jądro wielomianowe dziesiątego stopnia. Oczywiście jeśli model wykazuje cechy przetrenowania, powinniśmy zmniejszyć stopień wielomianu. Z kolei jeśli jest niedotrenowany, warto spróbować zwiększyć stopień wielomianu. Hiperparametr `coef0` reguluje proporcję wpływu wielomianów dużego stopnia do wielomianu małego stopnia na zachowanie modelu.



Rysunek 5.7. *Klasyfikatory SVM wykorzystujące jądro wielomianowe*



Powszechnie stosowanym sposobem wyszukiwania optymalnych wartości hiperparametrów jest metoda przeszukiwania siatki (zob. rozdział 2.). Często najszybciej jest przeprowadzić najpierw bardzo ogólne przeszukiwanie siatki, a następnie zwiększyć dokładność w okolicach najlepszych znalezionych wartości. Znajomość funkcji wykonywanej przez każdy hiperparametr również pozwala nam skoncentrować się na odpowiednim obszarze przestrzeni hiperparametrycznej.

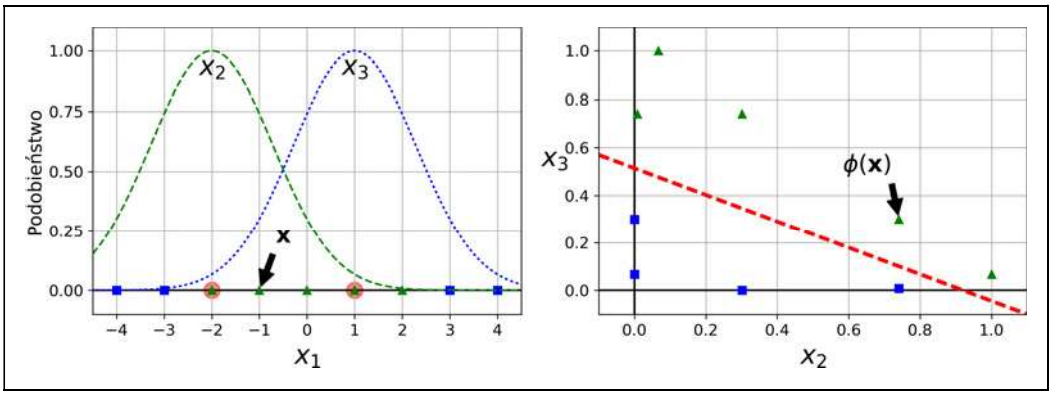
Cechy podobieństwa

Inną techniką pozwalającą na przetwarzanie problemów nieliniowych jest dodawanie cech wyliczonych za pomocą **funkcji podobieństwa** (ang. *similarity function*), sprawdzającej, w jakim stopniu dany przykład przypomina określony **punkt charakterystyczny** (ang. *landmark*). Na przykład weźmy wspomniany już jednowymiarowy zestaw danych i dodajmy do niego dwa punkty charakterystyczne $x_1 = -2$ i $x_1 = 1$ (lewy wykres na rysunku 5.8). Następnie na funkcję podobieństwa wyznaczmy gaussowską **radialną funkcję bazową** (ang. *radial basis function* — RBF) o parametrze $\gamma = 0{,}3$ (zob. równanie 5.1).

Równanie 5.1. Gaussowska funkcja RBF

$$\phi_{\gamma}(\mathbf{x}, \ell) = \exp\left(-\gamma\|\mathbf{x} - \ell\|^2\right)$$

Wykres tej funkcji ma dzwonowy kształt i przyjmuje wartości od 0 (daleko od punktu charakterystycznego) do 1 (dla punktu charakterystycznego). Teraz możemy obliczyć nowe cechy. Przykładowo, przyjrzyjmy się próbce $x_1 = -1$: dzieli ją odległość 1 od pierwszego punktu charakterystycznego i 2 od drugiego punktu charakterystycznego. Z tego powodu jego nowymi cechami są $x_2 = \exp(-0{,}3\times1^2) \approx 0{,}74$ i $x_3 = \exp(-0{,}3\times2^2) \approx 0{,}30$. Prawy wykres na rysunku 5.8 przedstawia już przekształcony zbiór danych (pozbawiony oryginalnych cech). Jak widać, teraz jest liniowo rozdzielny.



Rysunek 5.8. *Cechy podobieństwa uzyskane za pomocą gaussowskiej funkcji RBF*

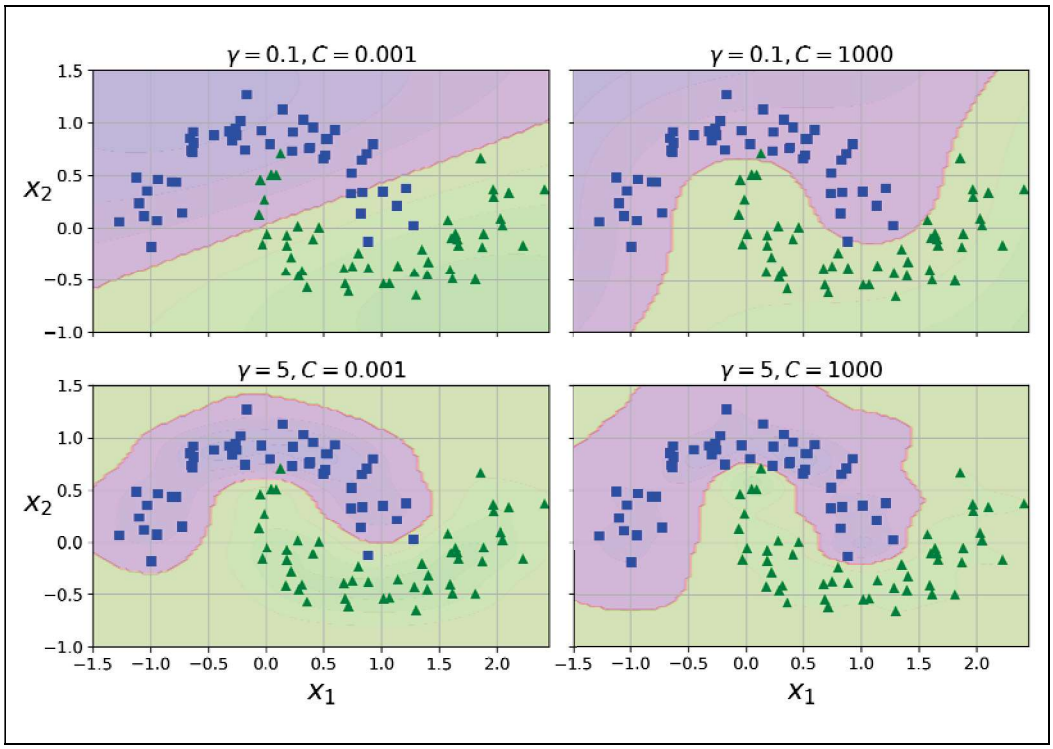
Zastanawiasz się pewnie, jak dobierać punkty charakterystyczne. Najprostszym rozwiązaniem jest ich utworzenie w lokacji każdego przykładu zbioru danych. W ten sposób uzyskujemy wiele wymiarów i zwiększamy szansę, że przekształcony zbiór danych będzie liniowo rozdzielny. Wadą tego rozwiązania jest fakt, że zbiór danych uczących zawierający m próbek i n cech zostaje przekształcony w zbiór uczący o m przykładach i m cechach (przy założeniu, że usuniemy pierwotne cechy). Jeśli korzystasz z bardzo dużego zestawu danych, wygenerujesz równie duży zbiór cech.

Gaussowskie jądro RBF

Podobnie jak w przypadku metody cech wielomianowych, technika cech podobieństwa okazuje się przydatna w każdym algorytmie uczenia maszynowego, ale wyliczenie wszystkich nowych cech może okazać się bardzo kosztowne obliczeniowo, zwłaszcza jeśli dysponujemy dużymi zbiorami danych. Również w tym przypadku sztuczka z jądrem okazuje się niezastąpiona: uzyskujemy za jej pomocą podobne wyniki, jak po dodaniu wielu cech. Sprawdźmy, co uzyskamy, implementując gaussowskie jądro RBF za pomocą klasy `SVC`:

```
rbf_kernel_svm_clf = Pipeline([
        ("scaler", StandardScaler()),
        ("svm_clf", SVC(kernel="rbf", gamma=5, C=0.001))
    ])
rbf_kernel_svm_clf.fit(X, y)
```

Model ten jest ukazany na lewym dolnym wykresie na rysunku 5.9. Pozostałe wykresy uzyskaliśmy, ucząc modele zawierające różne wartości hiperparametrów `gamma` (γ) i `C`. Wraz ze wzrostem wartości hiperparametru `gamma` krzywa dzwonowa staje się węższa (prawy wykres na rysunku 5.8). W konsekwencji zmniejsza się zakres wpływów każdej próbki: wynikowa granica decyzyjna jest bardziej nieregularna i zagina się przy każdym przykładzie. Z kolei mniejsze wartości hiperparametru `gamma` powodują rozszerzanie krzywej dzwonowej, dlatego próbki mają większą strefę wpływów, a granica decyzyjna cechuje się gładszym przebiegiem. Zatem γ pełni rolę hiperparametru regularyzacyjnego: jeżeli model ulega przetrenowaniu, należy zmniejszyć jego wartość, a jeśli jest niedotrenowany — zwiększyć (podobnie ma się rzecz w przypadku hiperparametru `C`).



Rysunek 5.9. *Klasyfikatory SVM wykorzystujące jądro RBF*



Przy takiej mnogości jąder jak należy wybrać właściwe? Zgodnie z niepisaną zasadą należy zawsze wypróbować najpierw jądro liniowe (pamiętaj, że klasa `LinearSVC` jest znacznie szybsza od jądra `SVC(kernel="linear")`), zwłaszcza jeśli zbiór danych jest bardzo duży lub zawiera wiele cech. Przy mniejszych rozmiarach zbioru uczącego warto sprawdzić również gaussowskie jądro RBF; w większości przypadków sprawdza się całkiem nieźle. Jeśli masz nadmiar czasu i mocy obliczeniowej, możesz poeksperymentować z kilkoma innymi jądrami, stosując metody sprawdzianu krzyżowego i przeszukiwania siatki. Dotyczy to przede wszystkim jąder dostosowanych do używanych przez Ciebie struktur danych.

Istnieją również inne jądra, ale są stosowane znacznie rzadziej. Niektóre są dostosowane do określonych struktur danych. **Jądra łańcuchowe** (ang. *string kernels*) są czasami używane do klasyfikowania dokumentów tekstowych lub sekwencji DNA (np. przy użyciu **jądra podsekwencji łańcucha znaków** — ang. *string subsequence kernel* — lub jąder wykorzystujących **odległość Levenstheina**).

Złożoność obliczeniowa

Klasa `LinearSVC` bazuje na bibliotece `LIBLINEAR`, zawierającej zoptymalizowany algorytm (*https://www.csie.ntu.edu.tw/~cjlin/papers/cddual.pdf*) liniowych maszyn SVM[1]. Nie obsługuje on sztuczki z jądrem, ale skaluje się niemal liniowo z liczbą przykładów uczących i liczbą cech. Jego złożoność czasu uczenia wynosi w przybliżeniu $O(m \times n)$.

Algorytm potrzebuje więcej czasu, jeśli wymagamy wyższej precyzji. Określamy ją za pomocą hiperparametru ϵ (w module Scikit-Learn nosi on nazwę `tol`). Dla większości zadań klasyfikacji domyślna wartość tolerancji jest wystarczająca.

Z kolei klasa `SVC` wywodzi się z biblioteki `LIBSVM`, implementującej algorytm (*https://homl.info/13*) wykorzystujący sztuczkę z jądrem[2]. Złożoność czasu uczenia mieści się tu zazwyczaj w przedziale pomiędzy $O(m^2 \times n)$ a $O(m^3 \times n)$. Niestety oznacza to, że algorytm przeraźliwie zwalnia w przypadku dużej liczby próbek uczących (np. setek tysięcy). Algorytm ten nadaje się znakomicie do skomplikowanych, niewielkich bądź średnich zbiorów uczących. Skaluje się on nieźle z liczbą cech, zwłaszcza jeśli są **rzadkie** (ang. *sparse features*; tj. gdy każdy przykład zawiera kilka niezerowych cech). W takiej sytuacji algorytm skaluje się w przybliżeniu z uśrednioną liczbą niezerowych cech przypadających na każdą próbkę. W tabeli 5.1 porównuję klasyfikatory SVM dostępne w module Scikit-Learn.

Tabela 5.1. Porównanie maszyn wektorów nośnych w module Scikit-Learn

Klasa	Złożoność obliczeniowa	Uczenie pozakorowe	Wymagane skalowanie	Sztuczka z jądrem
`LinearSVC`	$O(m \times n)$	Nie	Tak	Nie
`SGDClassifier`	$O(m \times n)$	Tak	Tak	Nie
`SVC`	$O(m^2 \times n)$ do $O(m^3 \times n)$	Nie	Tak	Tak

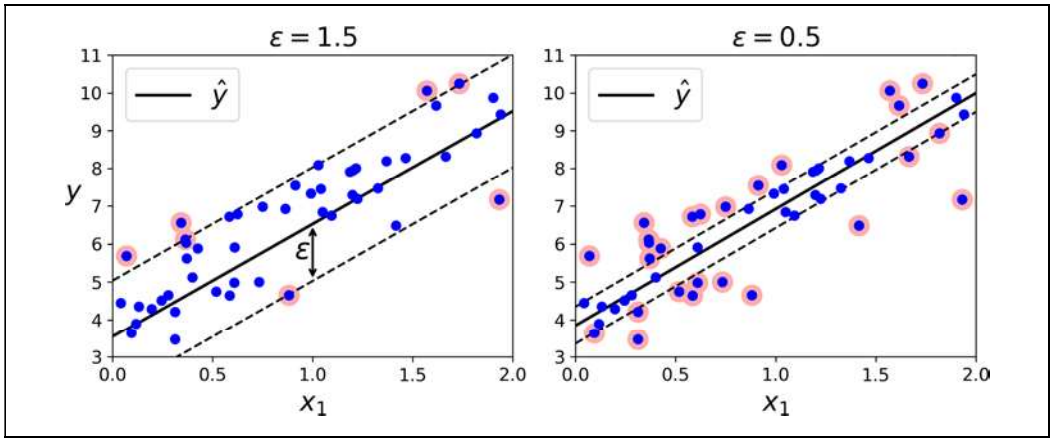
Regresja SVM

Jak już wspomniałem, algorytm SVM jest wszechstronny,ponieważ służy nie tylko do klasyfikacji liniowej i nieliniowej, ale obsługuje także regresję liniową/nieliniową. Tajemnica sukcesu wykorzystania maszyny SVM w zadaniach regresji, a nie klasyfikacji, tkwi w odwróceniu celu: w regresji SVM nie próbujemy uzyskać jak najszerszej „ulicy" pomiędzy dwiema klasami przy jednoczesnym ogra-

[1] Chih-Jen Lin i in., *A Dual Coordinate Descent Method for Large-Scale Linear SVM*, „Proceedings of the 25th International Conference on Machine Learning" (2008), s. 408 – 415.

[2] John Platt, *Sequential Minimal Optimization: A Fast Algorithm for Training Support Vector Machines*, „Microsoft Research technical report", April 21, 1998, *https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/tr-98-14.pdf*.

niczeniu naruszeń marginesu, lecz zmieścić jak najwięcej przykładów wewnątrz „ulicy” przy jak najmniejszej liczbie naruszeń marginesu (próbkach wykraczających poza jego granice). Szerokość marginesu jest regulowana za pomocą hiperparametru ϵ. Na rysunku 5.10 widzimy dwa liniowe modele regresji SVM wyuczone za pomocą losowych danych linowych; jeden model ma szeroki margines ($\epsilon = 1{,}5$), natomiast drugi ma niewielką tolerancję ($\epsilon = 0{,}5$).



Rysunek 5.10. *Regresja SVM*

Dodawanie przykładów wewnątrz marginesu nie wpływa na wyliczane prognozy; dlatego mówimy, że model ten jest *ϵ*-**nieczuły** (ang. *ϵ-insensitive*).

Możemy wykorzystać klasę `LinearSVR` do przeprowadzenia regresji SVM. Dzięki poniższemu kodowi uzyskujemy model zaprezentowany na lewym wykresie (rysunek 5.10; dane uczące powinny zostać najpierw wyskalowane i wyśrodkowane):

```
from sklearn.svm import LinearSVR

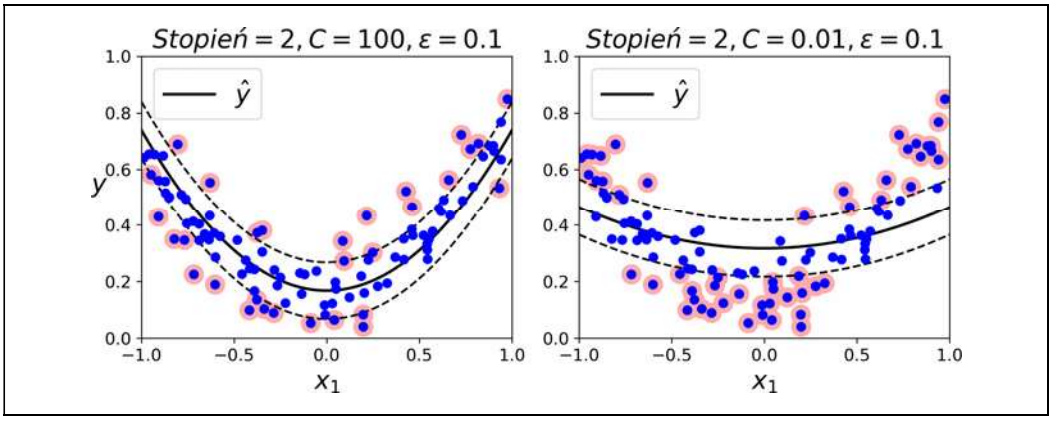
svm_reg = LinearSVR(epsilon=1.5)
svm_reg.fit(X, y)
```

Aby przetwarzać zadania regresji nieliniowej, możemy skorzystać z kernelizowanego modelu SVM. Na rysunku 5.11 ukazano model regresji SVM wobec losowego zbioru danych wielomianowych, wykorzystujący wielomianowe jądro drugiego stopnia. Na lewym wykresie wprowadzamy niewielką regularyzację (tj. dużą wartość hiperparametru `C`), natomiast na prawym wykresie stopień regularyzacji jest znacznie większy (mała wartość `C`).

Poniższy kod wykorzystuje klasę `SVR` (obsługującą sztuczkę z jądrem) do uzyskania modelu zaprezentowanego po lewej stronie na rysunku 5.11:

```
from sklearn.svm import SVR

svm_poly_reg = SVR(kernel="poly", degree=2, C=100, epsilon=0.1)
svm_poly_reg.fit(X, y)
```



Rysunek 5.11. Regresja SVM przy użyciu wielomianowego jądra drugiego stopnia

Klasa `SVR` stanowi regresyjny równoważnik klasy `SVC`, podobnie jak klasa `LinearSVR` jest regresyjnym ekwiwalentem klasy `LinearSVC`. Klasa `LinearSVR` skaluje się liniowo z rozmiarem zbioru uczącego (podobnie jak klasa `LinearSVC`), podczas gdy klasa `SVR/SVC` znacznie spowalnia działanie wraz ze wzrostem rozmiarów zestawu danych uczących.



Możemy również używać maszyn SVM do wykrywania elementów odstających; szczegóły znajdziesz w dokumentacji modułu Scikit-Learn.

Mechanizm działania

W tym podrozdziale wyjaśnię mechanizm wyliczania prognoz przez maszyny SVM oraz zasadę działania ich algorytmów uczących, począwszy od liniowych klasyfikatorów SVM. Jeżeli dopiero zaczynasz przygodę z uczeniem maszynowym, możesz od razu przejść do podrozdziału z ćwiczeniami i wrócić tu, gdy już lepiej zapoznasz się z maszynami wektorów nośnych.

Jeszcze słowo wyjaśnienia na temat notacji. W rozdziale 4. korzystaliśmy z konwencji umieszczania wszystkich parametrów modelu w jednym wektorze $\boldsymbol{\theta}$, w tym punkt obciążenia θ_0 i początkowe wagi cech od θ_1 do θ_n, a następnie dodawania wejściowego obciążenia $x_0 = 1$ do wszystkich próbek. W tym rozdziale użyjemy wygodniejszej (i powszechniejszej) konwencji dotyczącej maszyn SVM: punkt obciążenia będzie symbolizowany literą b, natomiast wektor wag cech przyjmie postać litery $\boldsymbol{w}$. Nie będziemy dodawać cechy obciążenia do wektora cech wejściowych.

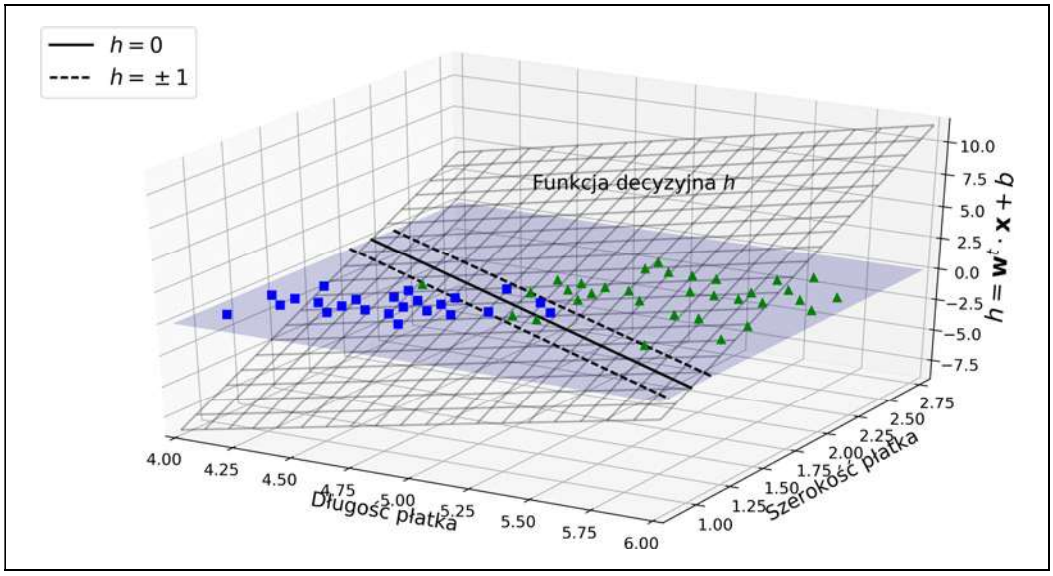
Funkcja decyzyjna i prognozy

Model liniowego klasyfikatora SVM przewiduje klasę nowego przykładu $\boldsymbol{x}$ poprzez obliczenie funkcji decyzyjnej $\mathbf{w}^T\mathbf{x} + b = w_1x_1 + \cdots + w_nx_n + b$. Jeżeli wynik jest dodatni, przewidywana klasa $\hat{y}$ będzie pozytywna (1), w przeciwnym razie zostanie prognozowana klasa negatywna (0) (równanie 5.2).

Równanie 5.2. Prognoza liniowego klasyfikatora SVM

$$\hat{y} = \begin{cases} 0 \ \textit{jeśli} \ \mathbf{w}^T \mathbf{x} + b < 0 \\ 1 \ \textit{jeśli} \ \mathbf{w}^T \mathbf{x} + b \geq 0 \end{cases}$$

Rysunek 5.12 prezentuje funkcję decyzyjną odpowiadającą modelowi ukazanemu na lewym wykresie na rysunku 5.4: jest to płaszczyzna dwuwymiarowa, ponieważ zbiór danych zawiera dwie cechy (długość i szerokość płatka). Granicę decyzyjną stanowi zbiór punktów, dla których wartość funkcji decyzyjnej wynosi 0: mieści się ona na przecięciu dwóch płaszczyzn i ma postać linii prostej (na rysunku jest ona symbolizowana przez pogrubioną linię ciągłą)[3].



Rysunek 5.12. *Funkcja decyzyjna dla zbioru danych Iris*

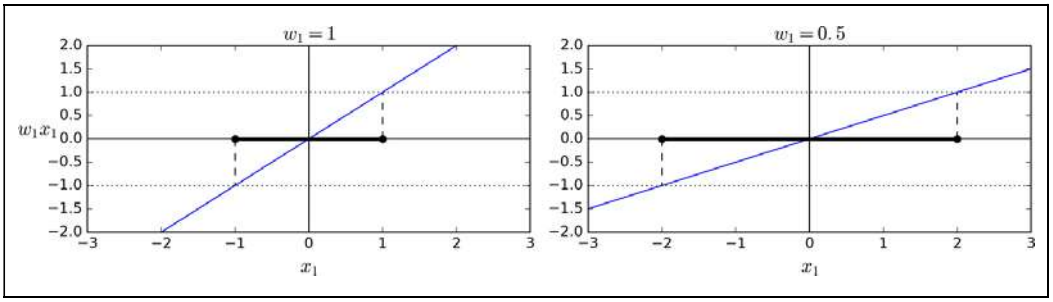
Linie przerywane ukazują punkty, w których funkcja decyzyjna przyjmuje wartości 1 lub –1: są one ułożone równolegle wobec granicy decyzyjnej i oddalone od niej o taką samą odległość — jest to nasz margines. Uczenie klasyfikatora SVM oznacza znalezienie wartości $\boldsymbol{w}$ i b pozwalających uzyskać jak najszerszy margines przy unikaniu jego naruszeń (twardy margines) lub ich ograniczeniu (miękki margines).

Cel uczenia

Rozważmy nachylenie funkcji decyzyjnej: jest ona równa normie wektora wag — $\|\mathbf{w}\|$. Jeżeli podzielimy wartości nachylenia przez 2, punkty, dla których wartości funkcji decyzyjnej będą równe ±1, będą znajdować się dwukrotnie dalej od granicy decyzyjnej. Innymi słowy, podzielenie nachylenia

[3] W bardziej ogólnym ujęciu, jeżeli mamy n cech, funkcja decyzyjna przyjmuje postać n-wymiarowej **hiperpłaszczyzny**, natomiast granica decyzyjna — hiperpłaszczyzny $(n–1)$-wymiarowej.

przez 2 spowoduje dwukrotne rozszerzenie marginesu. Zjawisko to zostało zilustrowane na rysunku 5.13. Im mniejszy wektor wag ***w***, tym uzyskujemy szerszy margines.



Rysunek 5.13. *Zmniejszenie wartości wektora wag powoduje rozszerzenie marginesu*

Chcemy zatem zminimalizować $\|\mathbf{w}\|$, aby uzyskać duży margines. Jeśli pragniemy uniknąć jego naruszeń (twardy margines), musimy sprawić, żeby funkcja decyzyjna miała wartości większe od 1 dla wszystkich pozytywnych próbek uczących, a mniejsze od –1 dla przykładów negatywnych. Jeśli zdefiniujemy $t^{(i)} = -1$ dla próbek negatywnych ($y^{(i)} = 0$), a $t^{(i)} = 1$ dla przykładów pozytywnych ($y^{(i)} = 1$), to możemy wyrazić to ograniczenie jako $t^{(i)}\left(\mathbf{w}^T\mathbf{x}^{(i)} + b\right) \geq 1$ dla wszystkich przykładów.

Możemy zatem odzwierciedlić cel liniowego klasyfikatora SVM wykorzystującego twardy margines jako problem **ograniczonej optymalizacji** (ang. *constrained optimization*) zaprezentowany w równaniu 5.3.

Równanie 5.3. Cel liniowego klasyfikatora SVM wykorzystującego twardy margines

$$\underset{\mathbf{w},b}{\text{minimalizuj}} \quad \frac{1}{2}\mathbf{w}^T\mathbf{w}$$

$$\text{pod warunkiem, że} \quad t^{(i)}\left(\mathbf{w}^T\mathbf{x}^{(i)} + b\right) \geq 1 \quad dla \quad i = 1, 2, \cdots, n$$



Nie minimalizujemy wyrażenia $\|\mathbf{w}\|$, lecz $\frac{1}{2}\mathbf{w}^T \cdot \mathbf{w}$, które jest równe $\frac{1}{2}\|\mathbf{w}\|^2$. Istotnie, wyrażenie $\frac{1}{2}\|\mathbf{w}\|^2$ ma całkiem prostą i elegancką pochodną (po prostu ***w***), podczas gdy $\|\mathbf{w}\|$ nie jest różniczkowalne w punkcie ***w*** = 0. Algorytmy optymalizacyjne działają znacznie sprawniej wobec różniczkowalnych funkcji.

Aby uzyskać cel miękkiego marginesu, musimy dla każdej próbki wprowadzić **zmienną swobodną** (ang. *slack variable*) $\zeta^{(i)} \geq 0$:[4] $\zeta^{(i)}$ definiuje stopień dopuszczalnego naruszenia marginesu przez i-ty przykład. Mamy teraz do czynienia z dwoma sprzecznymi celami: wyznaczeniem jak najmniejszych zmiennych swobodnych w celu ograniczenia naruszeń marginesu oraz zminimalizowanie $\frac{1}{2}\mathbf{w}^T \cdot \mathbf{w}$ po to, aby rozszerzyć margines. Właśnie tutaj przydaje się hiperparametr `C`: za jego pomocą jesteśmy

[4] Zeta (ζ) jest ósmą literą alfabetu greckiego.

w stanie wyznaczyć kompromis pomiędzy obydwoma celami. Teraz możemy zapisać problem ograniczonej optymalizacji (równanie 5.4).

Równanie 5.4. Cel liniowego klasyfikatora SVM wykorzystującego miękki margines

$$\underset{\mathbf{w},b,\zeta}{\text{minimalizuj}} \quad \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{m}\zeta^{(i)}$$

$$\text{pod warunkiem, że} \quad t^{(i)}\left(\mathbf{w}^T\mathbf{x}^{(i)} + b\right) \geq 1 \quad i \quad \zeta^{(i)} \geq 0 \quad dla \quad i = 1, 2, \cdots, m$$

Programowanie kwadratowe

Problemy twardego i miękkiego marginesu należą do dziedziny problemów optymalizacji funkcji kwadratowych z ograniczeniami liniowymi. Problemy tego są znane jako problemy **programowania kwadratowego** (ang. *quadratic programming* — QP). Istnieje wiele mechanizmów ich rozwiązywania wykorzystujących techniki, których omówienie wykracza poza zakres niniejszej książki[5]. Ogólny problem został sformułowany w równaniu 5.5.

Równanie 5.5. Problem programowania kwadratowego

$$\underset{\mathbf{p}}{\text{minimalizuj}} \quad \frac{1}{2}\mathbf{p}^T\mathbf{H}\mathbf{p} + \mathbf{f}^T\mathbf{p}$$

$$\text{pod warunkiem, że} \quad \mathbf{A}\mathbf{p} \leq \mathbf{b}$$

gdzie:

p jest n_p-wymiarowym wektorem (n_p = liczba parametrów),

H jest macierzą $n_p \times n_p$,

f jest n_p-wymiarowym wektorem,

A jest macierzą $n_c \times n_p$ (n_c = liczba ograniczeń),

b jest n_c-wymiarowym wektorem.

Zwróć uwagę, że wyrażenie $\mathbf{A}\mathbf{p} \leq \mathbf{b}$ w rzeczywistości definiuje ograniczenia n_c: $\mathbf{p}^T\mathbf{a}^{(i)} \leq b^{(i)}$ dla $i = 1, 2, \ldots, n_c$, gdzie $\boldsymbol{a}^{(i)}$ stanowi wektor zawierający elementy i-tego rzędu macierzy ***A***, natomiast $b^{(i)}$ jest i-tym elementem wektora ***b***.

Możemy łatwo sprawdzić, że po wyznaczeniu parametrów QP w poniższy sposób otrzymamy cel liniowego klasyfikatora SVM wykorzystującego twardy margines:

[5] Naukę programowania kwadratowego możesz rozpocząć od przeczytania książki *Convex Optimization* (*https://web.stanford.edu/~boyd/cvxbook/bv_cvxbook.pdf*) autorstwa Stephena Boyda i Lievena Vandenberge'a (Cambridge University Press, 2004) lub obejrzenia serii wykładów Richarda Browna (*https://www.youtube.com/watch?v=elw1z1L_KJc&index=1&list=PLh464gFUoJWOmBYla3zbZbc4nv2AXez6X*). Po polsku natomiast warto zapoznać się z pozycjami: *Analiza wypukła*, L. Kowalski i A. Piskorek (Wojskowa Akademia Techniczna) oraz *Wstęp do analizy wypukłej*, A. Piskorek (Wojskowa Akademia Techniczna).

- $n_p = n+1$, gdzie n stanowi liczbę cech (wartość +1 definiuje nasz punkt obciążenia),
- $n_c = m$, gdzie m stanowi liczbę próbek uczących,
- $\boldsymbol{H}$ stanowi macierz jednostkową $n_p \times n_p$, jedyne 0 znajduje się w pierwszej komórce (ignorujemy punkt obciążenia),
- $\boldsymbol{f} = 0$ jest n_p-wymiarowym wektorem wypełnionym zerami,
- $\boldsymbol{b} = -1$ jest n_c-wymiarowym wektorem wypełnionym ujemnymi jedynkami,
- $\mathbf{a}^{(i)} = -\mathbf{t}^{(i)}\dot{\mathbf{x}}^{(i)}$, gdzie $\dot{\mathbf{x}}^{(i)}$ jest równoważny wektorowi $\boldsymbol{x}^{(i)}$ zawierającemu dodatkową cechę obciążenia $\dot{\mathbf{x}}_0 = 1$.

Jednym ze sposobów trenowania liniowego klasyfikatora SVM wykorzystującego twardy margines jest zastosowanie dostępnego mechanizmu rozwiązywania QP i przekazanie mu powyższych parametrów. Wynikowy wektor $\boldsymbol{p}$ będzie zawierał punkt obciążenia $b = p_0$ oraz wagi cech $w_i = p_i$ dla $i = 1, 2, \ldots, n$. W analogiczny sposób możemy rozwiązywać problem miękkiego marginesu (zob. ćwiczenia na końcu rozdziału).

Żeby móc korzystać ze sztuczki z jądrem, musimy przeanalizować inny problem ograniczonej optymalizacji.

Problem dualny

Jeżeli znany jest problem ograniczonej optymalizacji, tzw. **problem pierwotny** (ang. *primal problem*), możliwe staje się wyrażenie odmiennego, ale blisko związanego zagadnienia, zwanego **problemem dualnym** (ang. *dual problem*). Rozwiązanie problemu dualnego zazwyczaj stanowi ograniczenie dolne rozwiązania problemu pierwotnego, jednak w pewnych warunkach rozwiązanie obydwu problemów może być takie samo. Na szczęście maszyny SVM spełniają te warunki[6], zatem możesz wybrać, czy chcesz rozwiązać problem pierwotny, czy dualny; w każdym przypadku uzyskasz to samo rozwiązanie. Równanie 5.6 przedstawia postać dualną celu liniowej maszyny SVM (jeżeli interesuje Cię wyprowadzenie formy dualnej z postaci pierwotnej, zajrzyj do dodatku C).

Równanie 5.6. Postać dualna celu liniowej maszyny SVM

$$\underset{\alpha}{\text{minimalizuj}} = \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}\alpha^{(i)}\alpha^{(j)}t^{(i)}t^{(j)}\mathbf{x}^{(i)T}\mathbf{x}^{(j)} - \sum_{i=1}^{m}\alpha^{(i)}$$

$$\text{pod warunkiem, że} \quad \alpha^{(i)} \geq 0 \quad dla \quad i = 1, 2, \cdots, m$$

Po wyznaczeniu wektora $\hat{\alpha}$ minimalizującego wynik powyższego równania (za pomocą mechanizmu rozwiązującego QP) wylicz za pomocą równania 5.7 wartości $\hat{\mathbf{w}}$ i $\hat{b}$ minimalizujące problem pierwotny.

[6] Funkcja celu jest wypukła, a ograniczenia nierówności są różniczkowalne w sposób ciągły i również stanowią funkcje wypukłe.

Równanie 5.7. Przejście od problemu dualnego do pierwotnego

$$\hat{\mathbf{w}} = \sum_{i=1}^{m} \hat{\alpha}^{(i)} t^{(i)} \mathbf{x}^{(i)}$$

$$\hat{b} = \frac{1}{n_s} \sum_{\substack{i=1 \\ \hat{\alpha}^{(i)} > 0}}^{m} \left(t^{(i)} - \hat{\mathbf{w}}^T \mathbf{x}^{(i)} \right)$$

Jeżeli liczba przykładów uczących jest mniejsza od liczby cech, problem dualny jest rozwiązywany szybciej od problemu pierwotnego. Co więcej, w przeciwieństwie do problemu pierwotnego, problem dualny pozwala nam korzystać ze sztuczki z jądrem. Czym więc jest właściwie ta sztuczka?

Kernelizowane maszyny SVM

Załóżmy, że chcemy zastosować transformację wielomianową drugiego stopnia wobec dwuwymiarowego zbioru danych uczących (np. zbioru danych księżycowatych), a następnie wytrenować na nim liniowy klasyfikator SVM. Równanie 5.8 ukazuje wielomianową funkcję drugiego stopnia ϕ, którą chcemy wykorzystać.

Równanie 5.8. Odwzorowanie wielomianowe drugiego stopnia

$$\phi(\mathbf{x}) = \phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \begin{pmatrix} {x_1}^2 \\ \sqrt{2} x_1 x_2 \\ {x_2}^2 \end{pmatrix}$$

Zwróć uwagę, że przekształcony wektor stał się trójwymiarowy. Zobaczmy teraz, co się stanie, jeśli przekształcimy w ten sposób dwa dwuwymiarowe wektory, ***a*** i ***b***, a następnie wyliczymy ich iloczyn skalarny[7] (równanie 5.9).

Równanie 5.9. Sztuczka z jądrem dla odwzorowania wielomianowego drugiego stopnia

$$\phi(\mathbf{a})^T \phi(\mathbf{b}) = \begin{pmatrix} {a_1}^2 \\ \sqrt{2} a_1 a_2 \\ {a_2}^2 \end{pmatrix}^T \begin{pmatrix} {b_1}^2 \\ \sqrt{2} b_1 b_2 \\ {b_2}^2 \end{pmatrix} = {a_1}^2 {b_1}^2 + 2 a_1 b_1 a_2 b_2 + {a_2}^2 {b_2}^2 =$$

$$= \left(a_1 b_1 + a_2 b_2\right)^2 = \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^T \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right)^2 = \left(\mathbf{a}^T \cdot \mathbf{b}\right)^2$$

[7] Jak już wyjaśniłem w rozdziale 4., iloczyn skalarny wektorów ***a*** i ***b*** zapisujemy zazwyczaj jako $\mathbf{a} \cdot \mathbf{b}$. Jednakże w uczeniu maszynowym wektory są często reprezentowane w postaci kolumnowej (tzn. jako macierze jednokolumnowe), dlatego iloczyn skalarny przyjmuje postać $\mathbf{a}^T\mathbf{b}$. Dla zachowania spójności z resztą książki będziemy korzystać tu ze wspomnianej notacji i przymkniemy oko na fakt, że z technicznego punktu widzenia otrzymujemy w ten sposób macierz jednoelementową, a nie wartość skalarną.

I jak? Iloczyn skalarny dwóch przekształconych wektorów jest równy kwadratowi iloczynu skalarnego dwóch pierwotnych wektorów: $\phi(\mathbf{a})^T \phi(\mathbf{b}) = (\mathbf{a}^T\mathbf{b})^2$.

Teraz nadszedł czas na ważne spostrzeżenie: jeżeli zastosujesz transformację ϕ wobec wszystkich próbek uczących, to problem dualny (równanie 5.6) będzie zawierał iloczyn skalarny $\phi(\mathbf{x}^{(i)})^T \phi(\mathbf{x}^{(j)})$. Jeśli jednak funkcja ϕ stanowi transformację wielomianową drugiego stopnia (równanie 5.8), to możemy zastąpić iloczyn skalarny tych przekształconych wektorów wyrażeniem $(\mathbf{x}^{(i)T}\mathbf{x}^{(j)})^2$. Zatem nie musimy wcale przekształcać przykładów uczących: wystarczy w równaniu 5.6 zastąpić iloczyn skalarny jego kwadratem. Otrzymasz dokładnie taki sam wynik jak w przypadku mozolnego przekształcania danych uczących i uczenia modelu za pomocą liniowego algorytmu SVM, jednak sztuczka ta jest znacznie oszczędniejsza obliczeniowo.

Funkcja $K(\mathbf{a},\mathbf{b}) = (\mathbf{a}^T\mathbf{b})^2$ jest nazywana **jądrem wielomianowym drugiego stopnia** (ang. *2nd-degree polynomial kernel*). W świecie uczenia maszynowego jądrem jest funkcja zdolna do obliczania iloczynu skalarnego $\phi(\mathbf{a})^T \phi(\mathbf{b})$ wyłącznie przy użyciu pierwotnych wektorów ***a*** i ***b***, w najmniejszym nawet stopniu niewykorzystująca przekształcenia ϕ. Równanie 5.10 zawiera listę najczęściej stosowanych jąder.

Równanie 5.10. Najpopularniejsze jądra

Liniowe: $K(\mathbf{a},\mathbf{b}) = \mathbf{a}^T\mathbf{b}$

Wielomianowe: $K(\mathbf{a},\mathbf{b}) = (\gamma\mathbf{a}^T\mathbf{b} + r)^d$

Gaussowskie RBF: $K(\mathbf{a},\mathbf{b}) = \exp(-\gamma\|\mathbf{a}-\mathbf{b}\|^2)$

Sigmoidalne: $K(\mathbf{a},\mathbf{b}) = \tanh(\gamma\mathbf{a}^T\mathbf{b} + r)$

Twierdzenie Mercera

Zgodnie z **twierdzeniem Mercera** jeżeli funkcja $K(\boldsymbol{a}, \boldsymbol{b})$ spełnia kilka matematycznych warunków, zwanych **warunkami Mercera** (tzn. funkcja K musi być ciągła i symetryczna tak, że $K(\boldsymbol{a}, \boldsymbol{b}) = K(\boldsymbol{b}, \boldsymbol{a})$ itd.), to istnieje dla niej funkcja ϕ mapująca wektory a i b na inną przestrzeń (może mieć ona większą liczbę wymiarów) tak, że $K(\mathbf{a},\mathbf{b}) = \phi(\mathbf{a})^T \phi(\mathbf{b})$. Zatem możemy korzystać z funkcji K jak z jądra, skoro wiemy, że istnieje dla niej przekształcenie ϕ, chociaż nawet nie musimy go znać. W przypadku gaussowskiego jądra RBF można udowodnić, że funkcja ϕ odwzorowuje każdą próbkę uczącą na przestrzeni o nieskończonej liczbie wymiarów, zatem bardzo dobrze, że nie musimy przeprowadzać takiego mapowania!

Zwróć uwagę, że pewne powszechnie stosowane jądra (np. sigmoidalne) nie spełniają wszystkich warunków Mercera, co nie przeszkadza im w większości przypadków w prawidłowym działaniu.

Pozostała nam jeszcze jedna sprawa do wyjaśnienia. Równanie 5.7 przedstawia sposób przejścia od rozwiązania dualnego do pierwotnego w przypadku liniowego klasyfikatora SVM. Jeśli jednak zastosujemy sztuczkę z jądrem, otrzymamy równania zawierające przekształcenie $\phi(x^{(i)})$. Wektor $\hat{\mathbf{w}}$ musi zawierać taką liczbę wymiarów, jak funkcja $\phi(x^{(i)})$, a może ona być olbrzymia albo wręcz nieskończona, zatem nie będziemy w stanie go wyliczyć. Jak jednak możemy obliczać prognozy bez wektora $\hat{\mathbf{w}}$? Dobra wiadomość jest taka, że możemy dołączyć wzór na wektor $\hat{\mathbf{w}}$ z równania 5.7 do funkcji decyzyjnej dla nowego przykładu $\boldsymbol{x}^{(n)}$, dzięki czemu otrzymamy równanie zawierające wyłącznie iloczyny skalarne wektorów wejściowych. W ten sposób możemy skorzystać ze sztuczki z jądrem (równanie 5.11).

Równanie 5.11. Wyliczanie prognoz za pomocą kernelizowanej maszyny SVM

$$h_{\hat{\mathbf{w}},\hat{b}}\left(\boldsymbol{\phi}\left(\mathbf{x}^{(n)}\right)\right) = \hat{\mathbf{w}}^T\boldsymbol{\phi}\left(\mathbf{x}^{(n)}\right) + \hat{b} = \left(\sum_{i=1}^{m}\hat{\alpha}^{(i)}t^{(i)}\boldsymbol{\phi}\left(\mathbf{x}^{(i)}\right)\right)^T \boldsymbol{\phi}\left(\mathbf{x}^{(n)}\right) + \hat{b} =$$

$$= \sum_{i=1}^{m}\hat{\alpha}^{(i)}t^{(i)}\left(\boldsymbol{\phi}\left(\mathbf{x}^{(i)}\right)^T\boldsymbol{\phi}\left(\mathbf{x}^{(n)}\right)\right) + \hat{b} =$$

$$= \sum_{\substack{i=1\\ \hat{\alpha}^{(i)}>0}}^{m}\hat{\alpha}^{(i)}t^{(i)}K\left(\mathbf{x}^{(i)},\mathbf{x}^{(n)}\right) + \hat{b}$$

Zauważ, że skoro $\alpha^{(i)} \neq 0$ wyłącznie dla wektorów nośnych, wyliczanie prognoz jest przeprowadzane za pomocą iloczynu skalarnego nowego wektora wejściowego $\boldsymbol{x}^{(n)}$ wraz z wyłącznie wektorami nośnymi, a nie wszystkimi próbkami uczącymi. Oczywiście za pomocą tej samej sztuczki musimy obliczyć punkt obciążenia $\hat{b}$ (równanie 5.12).

Równanie 5.12. Obliczanie punktu obciążenia za pomocą sztuczki z jądrem

$$\hat{b} = \frac{1}{n_s}\sum_{\substack{i=1\\ \hat{\alpha}^{(i)}>0}}^{m}\left(1 - t^{(i)}\hat{\mathbf{w}}^T\boldsymbol{\phi}\left(\mathbf{x}^{(i)}\right)\right) = \frac{1}{n_s}\sum_{\substack{i=1\\ \hat{\alpha}^{(i)}>0}}^{m}\left(t^{(i)} - \left(\sum_{j=1}^{m}\hat{\alpha}^{(j)}t^{(j)}\boldsymbol{\phi}\left(\mathbf{x}^{(j)}\right)\right)^T\boldsymbol{\phi}\left(\mathbf{x}^{(i)}\right)\right) =$$

$$= \frac{1}{n_s}\sum_{\substack{i=1\\ \hat{\alpha}^{(i)}>0}}^{m}\left(t^{(i)} - \sum_{\substack{j=1\\ \hat{\alpha}^{(j)}>0}}^{m}\hat{\alpha}^{(j)}t^{(j)}K\left(\mathbf{x}^{(i)},\mathbf{x}^{(j)}\right)\right)$$

Nie masz co się dziwić, jeżeli zaczyna boleć Cię głowa: taki jest nieszczęśliwy skutek uboczny sztuczki z jądrem.

Przyrostowe maszyny SVM

Zanim przejdziemy do następnego rozdziału, przyjrzyjmy się jeszcze pokrótce przyrostowym klasyfikatorom SVM (czyli takim, w których nauka odbywa się w czasie rzeczywistym, wraz z pojawianiem się nowych przykładów uczących).

W przypadku liniowych klasyfikatorów SVM jedną z metod implementacji przyrostowego klasyfikatora SVM jest użycie algorytmu gradientu prostego (np. za pomocą klasy `SGDClassifier`) w celu minimalizacji funkcji kosztu (równanie 5.13) — funkcja ta jest pochodną otrzymaną z problemu pierwotnego. Niestety algorytm gradientu prostego staje się zbieżny znacznie wolniej niż przy użyciu technik programowania kwadratowego.

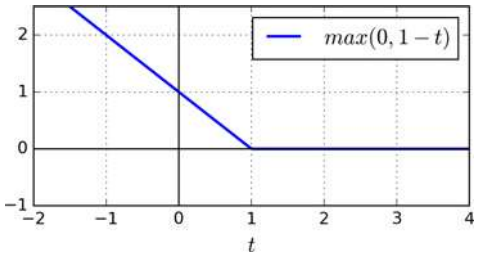
Równanie 5.13. Funkcja kosztu liniowego klasyfikatora SVM

$$J(\mathbf{w},b) = \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{m}\max\left(0, 1 - t^{(i)}\left(\mathbf{w}^T\mathbf{x}^{(i)} + b\right)\right)$$

Pierwsza suma w funkcji kosztu sprawia, że model będzie miał niewielki wektor wag *w*, co prowadzi do szerszego marginesu. Druga suma oblicza całkowitą wartość wszystkich naruszeń marginesu. Wartość naruszenia marginesu przez próbkę jest równa 0, jeżeli znajduje się poza nim i po właściwej stronie, w przeciwnym wypadku jest proporcjonalna do odległości od prawidłowej strony „ulicy". Poprzez minimalizowanie tego członu staramy się, aby naruszenia marginesu były jak najmniejsze i najrzadsze.

Zawiasowa funkcja straty

Funkcja $max(0, 1-t)$ jest nazywana **zawiasową funkcją straty** (została ukazana na poniższym wykresie). Jest równa 0, gdy $t \geq 1$. Jej pochodna (nachylenie) wynosi –1, jeśli $t < 1$, a 0, jeśli $t > 1$. Nie jest różniczkowalna w $t = 1$, ale podobnie jak w regresji metodą LASSO (zob. „Regresja metodą LASSO" w rozdziale 4.) jesteś w stanie korzystać z algorytmu gradientu prostego za pomocą dowolnej **subróżniczki** w $t = 1$ (tzn. dowolnej wartości pomiędzy –1 a 0).



Możliwa jest również implementacja innych przyrostowych, kernelizowanych maszyn SVM — co zostało opisane w publikacjach *Incremental and Decremental SVM Learning*[8] (*https://papers.nips.*

[8] Gert Cauwenberghs i Tomaso Poggio, *Incremental and Decremental Support Vector Machine Learning*, „Proceedings of the 13th International Conference on Neural Information Processing Systems" (2000), s. 388 – 394.

cc/paper/1814-incremental-and-decremental-support-vector-machine-learning.pdf) i *Fast Kernel Classifiers with Online and Active Learning*[9] (*http://www.jmlr.org/papers/volume6/bordes05a/bordes05a.pdf*). Wymienione modele są jednak zaimplementowane w środowiskach Matlab i C++. W przypadku wielkoskalowych problemów nieliniowych lepiej skorzystać z sieci neuronowych (część II).

Ćwiczenia

1. Jaka jest podstawowa koncepcja maszyn wektorów nośnych?
2. Czym jest wektor nośny?
3. Dlaczego należy skalować dane wejściowe podczas korzystania z maszyn SVM?
4. Czy klasyfikator SVM może obliczać wynik pewności podczas klasyfikowania przykładu? Czy może wyliczać prawdopodobieństwo przynależności próbki do danej klasy?
5. Z jakiego rozwiązania (pierwotnego czy dualnego) należy korzystać podczas uczenia modelu maszyny SVM wobec zestawu danych składającego się z milionów przykładów i setek cech?
6. Załóżmy, że wytrenowaliśmy klasyfikator SVM za pomocą jądra RBF, ale wydaje się, że jest niedotrenowany. Powinniśmy zwiększyć czy zmniejszyć wartość hiperparametru γ (`gamma`)? Co z hiperparametrem `C`?
7. Jak należy skonfigurować parametry programowania kwadratowego ($\boldsymbol{H}$, $\boldsymbol{f}$, $\boldsymbol{A}$ i $\boldsymbol{b}$) w celu użycia liniowego klasyfikatora SVM wykorzystującego miękki margines za pomocą dostępnego mechanizmu QP?
8. Wyucz klasę `LinearSVC` wobec zbioru liniowo rozdzielnych danych. Następnie wytrenuj na nim również klasy `SVC` i `SGDClassifier`. Spróbuj uzyskać za ich pomocą podobny model.
9. Wyucz klasyfikator `SVM` wobec zbioru danych MNIST. Klasyfikatory SVM są binarne, zatem musisz zastosować strategię OvR, żeby były rozpoznawane wszystkie cyfry. Możesz chcieć dostroić hiperparametry za pomocą niewielkich zestawów walidacyjnych, aby przyśpieszyć cały proces. Jaką dokładność jesteś w stanie osiągnąć?
10. Wytrenuj regresor `SVM` wobec zbioru danych *California housing*.

Rozwiązania i odpowiedzi znajdziesz w dodatku A

[9] Antoine Bordes i in., *Fast Kernel Classifiers with Online and Active Learning*, „Journal of Machine Learning Research" 6 (2005), s. 1579 – 1619.